

Lecture 8: Closest Pair of Points

Date: August 21, 2026
Instructor: Aniket Basu Roy
Scribe: Saketh S and Kalidas Jayakumar

Overview. Finding the minimum value in an unsorted set takes $O(n)$ time. Finding the minimum distance between points on a 1-dimensional line takes $O(n \log n)$ time via sorting. Given a set of points on a 2-Dimensional plane, we show how to find the closest pair of points efficiently in $O(n \log n)$ time using a divide-and-conquer strategy paired with a geometric packing argument.

1 Finding the Closest Pair of Points

Motivation. Consider a set of n points $P = \{p_1, \dots, p_n\}$, $p_i \in \mathbb{R}^2$, on a 2D plane. We want to find a pair of points (p^*, q^*) such that the Euclidean distance $d(p^*, q^*)$ is minimized across all distinct pairs.

Input: $P = \{p_1, \dots, p_n\}$, $p_i \in \mathbb{R}^2$

Output: $p^*, q^* \in P$ such that $d(p^*, q^*)$ is the minimum pairwise distance in P .

2 The Algorithm

Algorithm 1: Finding the Closest Pair of Points

Input: A set of points $P = \{p_1, \dots, p_n\} \subset \mathbb{R}^2$

Output: Points $p^*, q^* \in P$ achieving the minimum Euclidean distance

```

1 Function ClosestPair( $P$ ):
2   if  $|P| \leq 3$  then
3     Compute all pairwise distances by brute force and return  $(p^*, q^*)$ 
4   Divide  $P$  by vertical line  $L : x = x_{\text{mid}}$  into equal subsets  $Q$  and  $R$ 
5    $(q_1^*, q_2^*) \leftarrow \text{ClosestPair}(Q)$ 
6    $(r_1^*, r_2^*) \leftarrow \text{ClosestPair}(R)$ 
7    $\delta \leftarrow \min(d(q_1^*, q_2^*), d(r_1^*, r_2^*))$ 
8   Let  $(p^*, q^*)$  be the pair achieving distance  $\delta$ 
9    $S \leftarrow \{p \in P \mid |p.x - x_{\text{mid}}| < \delta\}$ , sorted by  $y$ -coordinate
10  for  $i \leftarrow 1$  to  $|S| - 1$  do
11    for  $j \leftarrow i + 1$  to  $\min(i + 15, |S|)$  do
12      if  $d(S[i], S[j]) < \delta$  then
13         $\delta \leftarrow d(S[i], S[j])$ 
14         $(p^*, q^*) \leftarrow (S[i], S[j])$ 
15  return  $(p^*, q^*)$ 

```

Algorithm 1 divides the point set into equal halves Q and R using a vertical line L . It computes the recursive closest pairs in both halves and sets $\delta = \min(\delta_Q, \delta_R)$. It then builds a strip S of width 2δ centered at L , sorts S by y -coordinate, and evaluates distances to at most 15 subsequent neighbors for each point.

3 Proof of Correctness

Claim 1. If (p, q) is the closest pair of points in P such that $p \in Q$, $q \in R$ and $d(p, q) < \delta$, then both p and q must lie in the strip S .

Proof. Suppose $p \in Q$, $q \in R$ and $d(p, q) < \delta$. If p or q is at a distance greater than δ from line L , then the distance between p and q will be at least δ . This contradicts our assumption that $d(p, q) < \delta$.

Therefore both p and q must lie in S . □

Claim 2. If the shortest distance pair is within strip S , then they are at most 15 positions apart in S_y .

Proof. Let S_y be the points in S sorted in increasing order of their y co-ordinates. We divide the strip S into boxes of size $\delta/2 \times \delta/2$. The maximum distance between any two points in the box is the diagonal of the box,

$$\sqrt{\left(\frac{\delta}{2}\right)^2 + \left(\frac{\delta}{2}\right)^2} = \frac{\delta}{\sqrt{2}} < \delta.$$

And since the box lie entirely in either Q or R there should be at most one point per box otherwise it contradicts the definition of δ .

Suppose two points are separated by 16 positions in S_y . Since each row contains 4 boxes, then there must be atleast 3 rows between them. Therefore vertical distance between them is,

$$3\frac{\delta}{2} > \delta.$$

Therefore, any two points in S whose distance is less than δ must be at most 15 positions apart in S_y . $\square$

Theorem 1. *The algorithm correctly returns the closest pair of points in P .*

Proof. We prove the result by induction. Let $|P| = n$.

Base Case: If $n \leq 3$, the algorithm checks all possible pairs and returns the pair with minimum distance. Therefore, it returns correct pair of points.

Induction Hypothesis: Assume that the algorithm correctly finds the closest pair for every set containing less than n points.

Induction Step: Consider a set P containing n points. The algorithm divides P into Q and R and recursively finds the closest pair in each set. By the induction hypothesis, the result returned by the recursive calls are correct. Let,

$$\delta = \min(\delta_Q, \delta_R).$$

Suppose (p, q) be the actual closest pair in P . Then there will be two cases.

Case 1: Both points are in the same half. If both p and q are in Q , or both are in R , then according to induction hypothesis the recursive calls will return closest pair.

Case 2: The points are on opposite sides of L . Suppose $p \in Q$ and $q \in R$. If $d(p, q) < \delta$, then by Claim 1 both points lie in the strip S . By Claim 2, they are at most 15 points apart in S_y . Therefore, the merging step checks the pair (p, q) and hence find the closest pair.

Thus the algorithm will find the closest pair of points in both the cases.

Hence the algorithm will always returns the closest pair of points. $\square$

4 Time Complexity

Let $T(n)$ be the time complexity for n points:

1. **Pre-sorting / Sublist creation:** Pre-sorting P by x - and y -coordinates takes $O(n \log n)$ time once. Maintaining y -sorted order during splitting takes $O(n)$ time per recursion step.
2. **Recursive calls:** Divided into 2 subproblems of size $n/2$, taking $2T(n/2)$ time.
3. **Combine step:** Filtering points into S takes $O(n)$ time. The inner loop evaluates at most 15 neighbors for each of the $|S| \leq n$ points, taking $15 \cdot O(n) = O(n)$ time.

This yields the recurrence relation:

$$T(n) = 2T(n/2) + O(n)$$

By Case 2 of the Master Theorem (or $O(n)$ work across $\log n$ recursion levels), $T(n) = O(n \log n)$.

Reference. Kleinberg, J., & Tardos, E. (2006). *Algorithm Design*, Chapter 5.4. Pearson/Addison-Wesley.